

ODC-GUINÉE

Formation Web Full-Stack — 8 avril au 25 juin 2026

Cahier de Conception Base de Données

MedConnect ODC — Plateforme de Prise de Rendez-vous Médical

Projet	Plateforme de prise de rendez-vous médical
Contexte	Projet professionnelle ODC-Guinée
SGBD cible	MySQL
Architecture	Découplée — API REST (Spring Boot / React)

Sommaire

1. Introduction et Objectifs
2. Règles de Gestion et Exigences Métier
3. Processus de Normalisation (1FN → 3FN)
4. Modèle Conceptuel de Données (MCD)
5. Modèle Logique de Données (MLD)
6. Modèle Physique de Données (MPD) — Script SQL
7. Intégrité Référentielle et Stratégie de Suppression
8. Conclusion

1. Introduction et Objectifs

Ce document constitue le cahier de conception technique de la base de données de la plateforme MedConnect ODC, une application web de gestion de rendez-vous médicaux développée dans le cadre du projet de fin de formation Full-Stack ODC-Guinée. Il décrit l'ensemble de la démarche de modélisation des données, depuis l'analyse des besoins métier jusqu'au script SQL de création physique de la base, en passant par le processus de normalisation et les modèles conceptuel et logique.

La base de données a été conçue pour répondre à un objectif central : digitaliser de façon fiable, sécurisée et traçable la gestion des rendez-vous médicaux entre patients et médecins, tout en garantissant l'intégrité référentielle des données et la conformité à la Troisième Forme Normale (3FN).

1.1. Objectifs de la conception

- Garantir l'intégrité et la cohérence des données grâce à un schéma relationnel normalisé.
- Supporter un modèle d'authentification unique et un système de rôles (PATIENT, MEDECIN, ADMIN).
- Permettre la traçabilité complète des rendez-vous, des consultations et des actions des utilisateurs (audit).
- Offrir des performances de lecture optimales grâce à un indexage ciblé sur les colonnes les plus sollicitées.
- Faciliter l'évolutivité du modèle (ajout de nouvelles spécialités, nouveaux types de notifications, etc.) sans remise en cause de la structure existante.

2. Règles de Gestion et Exigences Métier

L'application est soumise à des contraintes métier fortes, destinées à garantir la sécurité, l'auditabilité et la cohérence fonctionnelle exigées par le cahier des charges du projet :

2.1. Gestion des comptes et des rôles

- Inscription exclusive : seuls les utilisateurs souhaitant obtenir le rôle PATIENT peuvent s'inscrire librement sur la plateforme via le formulaire public. Un profil Patient leur est automatiquement associé en base de données lors de l'inscription.
- Création des médecins : les comptes de rôle MEDECIN sont exclusivement créés par un administrateur système authentifié, avec l'affectation obligatoire d'une spécialité médicale existante.
- Double sécurité administrateur : l'accès à l'interface d'administration requiert la validation des identifiants (email/mot de passe) couplée à la saisie obligatoire d'une clé secrète d'activation globale, vérifiée côté serveur.
- Désactivation réversible : un compte désactivé (`est_actif = FALSE`) conserve l'intégralité de son historique en base — aucune donnée n'est supprimée physiquement, afin de préserver la traçabilité.

2.2. Gestion des rendez-vous et consultations

- Unicité du créneau : une disponibilité ne peut être liée qu'à un seul rendez-vous actif à la fois (`est_libre` passe à `FALSE` dès la réservation afin de bloquer toute concurrence d'accès).
- Libération automatique : en cas d'annulation d'un rendez-vous (par le patient ou le médecin), le créneau de disponibilité associé est automatiquement remis en statut libre.
- Historique et traçabilité : dès qu'un rendez-vous est honoré, le médecin rédige une fiche de consultation qui stocke de façon définitive et immuable le diagnostic, les notes médicales et l'ordonnance. La suppression d'un rendez-vous ayant donné lieu à une consultation est interdite (`ON DELETE RESTRICT`) afin de préserver le dossier médical du patient.
- Supervision totale : l'administrateur dispose d'une visibilité à 360° sur l'ensemble des utilisateurs, des rendez-vous, des consultations et des notifications de la plateforme, à des fins de modération et de suivi global de l'activité.

3. Processus de Normalisation (1FN → 3FN)

La normalisation est le processus qui consiste à structurer les données afin d'éliminer les redondances, de prévenir les anomalies de mise à jour et de garantir l'intégrité du modèle relationnel. Le schéma de MedConnect ODC a été conduit jusqu'à la Troisième Forme Normale (3FN).

3.1. Première Forme Normale (1FN)

Règle : tout attribut doit être atomique (valeur unique, non décomposable) et chaque entité doit posséder une clé primaire unique permettant d'identifier sans ambiguïté chaque enregistrement.

Application : l'ensemble des attributs identifiés (nom, prénom, email, dates, textes de consultation, etc.) sont atomiques ; aucune colonne ne contient de liste ou de valeurs multiples. Chaque table dispose d'un identifiant technique auto-incrémenté (id_user, id_dispo, id_rdv, id_consultation, etc.) défini comme clé primaire.

3.2. Deuxième Forme Normale (2FN)

Règle : être en 1FN et garantir que tout attribut non clé dépend de la totalité de la clé primaire (pertinent uniquement pour les clés composées).

Application : l'ensemble des clés primaires retenues étant simples (non composées), chaque attribut non clé dépend par construction de l'intégralité de sa clé primaire correspondante. Aucune dépendance partielle n'est donc possible dans ce modèle.

3.3. Troisième Forme Normale (3FN)

Règle : être en 2FN et s'assurer qu'aucun attribut non clé ne dépend d'un autre attribut non clé (absence de dépendance transitive).

Application : les données relatives à l'historique médical (diagnostic, notes, ordonnance) et aux alertes applicatives ont été externalisées dans leurs propres entités dédiées (consultations et notifications), plutôt que d'être ajoutées comme colonnes nullables dans la table rendez_vous. Cette séparation évite les redondances, supprime le risque de valeurs nulles chroniques et isole chaque domaine fonctionnel dans une entité cohérente.

3.4. Stratégie d'héritage des profils utilisateurs

Pour répondre à l'exigence d'une base normalisée tout en conservant un modèle objet propre côté backend, le projet retient une stratégie d'héritage par jointure (table-per-subclass) : la table utilisateurs centralise les données communes d'authentification (identité, email, mot de passe haché, rôle, statut), tandis que les tables patients et medecins étendent cette entité mère par une relation 1:1 stricte (clé étrangère unique vers id_utilisateur), n'y ajoutant que les attributs spécifiques à chaque profil métier.

4. Modèle Conceptuel de Données (MCD)

Le MCD représente les entités du domaine métier ainsi que les associations et cardinalités qui les relient, indépendamment de toute contrainte technique d'implémentation.

4.1. Entités identifiées

Entité	Description
UTILISATEUR	Entité mère regroupant les informations communes d'authentification : identité, email, mot de passe, rôle, statut d'activité, date d'inscription.
PATIENT	Profil métier spécialisant un utilisateur de rôle PATIENT : coordonnées, antécédents médicaux.
MEDECIN	Profil métier spécialisant un utilisateur de rôle MEDECIN : matricule, coordonnées, spécialité de rattachement.
SPECIALITE	Référentiel des domaines d'expertise médicale (cardiologie, pédiatrie, dermatologie, etc.).
DISPONIBILITE	Créneau horaire ouvert par un médecin pour la consultation, libre ou réservé.
RENDEZ_VOUS	Réservation associant un patient à une disponibilité d'un médecin, avec un statut de suivi.
CONSULTATION	Fiche médicale rédigée à l'issue d'un rendez-vous honoré : diagnostic, notes, ordonnance.
NOTIFICATION	Alerte applicative adressée à un utilisateur (rappel, confirmation, information).
LOG	Trace d'audit des actions sensibles réalisées sur la plateforme.

4.2. Cardinalités et associations

- UTILISATEUR $\leftarrow$ (héritage par spécialisation) $\rightarrow$ PATIENT / MEDECIN : un utilisateur possède un rôle qui détermine son profil métier associé.
- SPECIALITE (0,N) — POSSEDER — (1,1) MEDECIN : une spécialité concerne zéro, un ou plusieurs médecins ; un médecin possède une seule spécialité principale.
- MEDECIN (0,N) — DECLARER — (1,1) DISPONIBILITE : un médecin déclare zéro, un ou plusieurs créneaux ; un créneau appartient à un seul médecin.
- PATIENT (0,N) — RESERVER — (1,1) RENDEZ_VOUS : un patient planifie zéro, un ou plusieurs rendez-vous ; un rendez-vous appartient à un seul patient.
- DISPONIBILITE (1,1) — CORRESPONDRE — (0,1) RENDEZ_VOUS : un créneau de disponibilité peut être réservé pour, au maximum, un seul rendez-vous.

- RENDEZ_VOUS (0,1) — GENERER — (1,1) CONSULTATION : un rendez-vous honoré génère au maximum une fiche de consultation unique.
- UTILISATEUR (1,1) — RECEVOIR — (0,N) NOTIFICATION : tout utilisateur peut recevoir zéro, une ou plusieurs notifications.
- UTILISATEUR (0,1) — GENERER — (0,N) LOG : un utilisateur peut être à l'origine de zéro, une ou plusieurs entrées de journal d'audit.

5. Modèle Logique de Données (MLD)

Le MLD traduit le MCD en un ensemble de relations (tables), avec leurs attributs, leurs clés primaires (soulignées) et leurs clés étrangères (préfixées par #), conformément aux règles de passage du modèle conceptuel vers le modèle relationnel.

utilisateurs (id_utilisateur, nom, prenom, email, mot_de_passe, id_role, est_actif, date_inscription, profile_image_url)
 roles (id_role, nom)
 specialites (id_specialite, nom, description)
 patients (id_patient, #id_utilisateur, telephone, adresse, antecedents_medicaux)
 medecins (id_medecin, #id_utilisateur, #id_specialite, telephone, adresse)
 disponibilites (id_dispo, #id_medecin, date_debut, date_fin, duree, est_libre)
 rendez_vous (id_rdv, #id_patient, #id_dispo, date_heure, duree, statut, motif)
 consultations (id_consultation, #id_rdv, date_consultation, diagnostic, notes_medicales, ordonnance)
 notifications (id_notification, #id_utilisateur, message, date_envoi, est_lu, type)
 logs (id_log, #id_utilisateur, action, details, date_action, ip_address)
 password_reset_tokens (id, #user_id, token, expiry_date)

5.1. Dictionnaire de données

Le tableau ci-dessous détaille les principaux attributs, leur type logique, leur caractère obligatoire et leur signification métier.

Table	Attribut	Type	Description
utilisateurs	id_utilisateur	BIGINT (PK)	Identifiant technique unique de l'utilisateur.
	email	VARCHAR(100)	Identifiant de connexion, unique sur l'ensemble de la plateforme.
	mot_de_passe	VARCHAR(255)	Empreinte BCrypt du mot de passe — jamais stocké en clair.
	id_role	BIGINT (FK)	Rôle de l'utilisateur : PATIENT, MEDECIN ou ADMIN.

Table	Attribut	Type	Description
	est_actif	BOOLEAN	Indique si le compte est actif ou désactivé par un administrateur.
medecins	id_specialite	BIGINT (FK)	Spécialité médicale principale du praticien.
disponibilites	est_libre	BOOLEAN	Indique si le créneau est encore disponible à la réservation.
disponibilites	duree	INT	Durée du créneau en minutes, calculée automatiquement.
rendez_vous	statut	ENUM	ATTENTE, CONFIRME, ANNULE ou TERMINE.
consultations	id_rdv	BIGINT (FK, UNIQUE)	Garantit la relation 1:1 stricte avec le rendez-vous honoré.
notifications	type	ENUM	INFO, ALERTE ou RAPPEL.
logs	ip_address	VARCHAR(45)	Adresse IP à l'origine de l'action, à des fins d'audit de sécurité.

6. Modèle Physique de Données (MPD) — Script SQL

Le script ci-dessous correspond à l'implémentation physique du modèle sur MySQL 8, incluant les contraintes d'intégrité référentielle, les contraintes de validation (CHECK), les index de performance et le moteur de stockage InnoDB pour le support transactionnel et les clés étrangères.

6.1. Tables de référence

```
CREATE TABLE specialites (  
  id_specialite BIGINT AUTO_INCREMENT PRIMARY KEY,  
  nom VARCHAR(100) NOT NULL UNIQUE,  
  description TEXT,  
  INDEX idx_specialite_nom (nom)  
) ENGINE=InnoDB;  
  
CREATE TABLE roles (  
  id_role BIGINT AUTO_INCREMENT PRIMARY KEY,  
  nom ENUM('PATIENT', 'MEDECIN', 'ADMIN') NOT NULL UNIQUE  
) ENGINE=InnoDB;
```

6.2. Table générique des utilisateurs

```
CREATE TABLE utilisateurs (  
  id_utilisateur BIGINT AUTO_INCREMENT PRIMARY KEY,  
  nom VARCHAR(50) NOT NULL,  
  prenom VARCHAR(50) NOT NULL,  
  email VARCHAR(100) NOT NULL UNIQUE,  
  mot_de_passe VARCHAR(255) NOT NULL,  
  id_role BIGINT NOT NULL,  
  est_actif BOOLEAN DEFAULT TRUE,  
  date_inscription DATETIME DEFAULT CURRENT_TIMESTAMP,  
  profile_image_url VARCHAR(255) DEFAULT NULL,  
  FOREIGN KEY (id_role) REFERENCES roles(id_role),  
  INDEX idx_utilisateur_email (email),  
  INDEX idx_utilisateur_role (id_role)  
) ENGINE=InnoDB;
```

6.3. Profils métier (héritage 1:1)

```
CREATE TABLE medecins (  
  id_medecin BIGINT AUTO_INCREMENT PRIMARY KEY,  
  id_utilisateur BIGINT NOT NULL UNIQUE,  
  id_specialite BIGINT,  
  telephone VARCHAR(20),  
  adresse TEXT,  
  FOREIGN KEY (id_utilisateur) REFERENCES utilisateurs(id_utilisateur) ON DELETE CASCADE,  
  FOREIGN KEY (id_specialite) REFERENCES specialites(id_specialite),  
  INDEX idx_medecin_specialite (id_specialite)  
) ENGINE=InnoDB;
```

```
CREATE TABLE patients (
  id_patient BIGINT AUTO_INCREMENT PRIMARY KEY,
  id_utilisateur BIGINT NOT NULL UNIQUE,
  telephone VARCHAR(20),
  adresse TEXT,
  antecedents_medicaux TEXT,
  FOREIGN KEY (id_utilisateur) REFERENCES utilisateurs(id_utilisateur) ON DELETE CASCADE
) ENGINE=InnoDB;
```

6.4. Disponibilités et rendez-vous

```
CREATE TABLE disponibilites (
  id_dispo BIGINT AUTO_INCREMENT PRIMARY KEY,
  id_medecin BIGINT NOT NULL,
  date_debut DATETIME NOT NULL,
  date_fin DATETIME NOT NULL,
  duree INT NOT NULL COMMENT 'Durée en minutes',
  est_libre BOOLEAN DEFAULT TRUE,
  FOREIGN KEY (id_medecin) REFERENCES medecins(id_medecin) ON DELETE CASCADE,
  CONSTRAINT chk_dates CHECK (date_fin > date_debut),
  INDEX idx_dispo_medecin (id_medecin),
  INDEX idx_dispo_date_debut (date_debut)
) ENGINE=InnoDB;
```

```
CREATE TABLE rendez_vous (
  id_rdv BIGINT AUTO_INCREMENT PRIMARY KEY,
  id_patient BIGINT NOT NULL,
  id_dispo BIGINT NOT NULL,
  date_heure DATETIME NOT NULL,
  duree INT NOT NULL COMMENT 'Durée en minutes',
  statut ENUM('ATTENTE', 'CONFIRME', 'ANNULE', 'TERMINE') DEFAULT 'ATTENTE',
  motif TEXT,
  FOREIGN KEY (id_patient) REFERENCES patients(id_patient) ON DELETE CASCADE,
  FOREIGN KEY (id_dispo) REFERENCES disponibilites(id_dispo) ON DELETE CASCADE,
  INDEX idx_rdv_patient (id_patient),
  INDEX idx_rdv_dispo (id_dispo),
  INDEX idx_rdv_date_heure (date_heure),
  CONSTRAINT chk_rdv_duree CHECK (duree > 0)
) ENGINE=InnoDB;
```

6.5. Consultations, notifications et audit

```
CREATE TABLE consultations (
  id_consultation BIGINT AUTO_INCREMENT PRIMARY KEY,
  id_rdv BIGINT NOT NULL UNIQUE,
  date_consultation DATETIME NOT NULL,
  diagnostic TEXT,
  notes_medicales TEXT,
  ordonnance TEXT,
  FOREIGN KEY (id_rdv) REFERENCES rendez_vous(id_rdv) ON DELETE RESTRICT,
  INDEX idx_consultation_rdv (id_rdv)
```

```
) ENGINE=InnoDB;
```

```
CREATE TABLE notifications (  
  id_notification BIGINT AUTO_INCREMENT PRIMARY KEY,  
  id_utilisateur BIGINT NOT NULL,  
  message VARCHAR(255) NOT NULL,  
  date_envoi DATETIME DEFAULT CURRENT_TIMESTAMP,  
  est_lu BOOLEAN DEFAULT FALSE,  
  type ENUM('INFO', 'ALERTE', 'RAPPEL') DEFAULT 'INFO',  
  FOREIGN KEY (id_utilisateur) REFERENCES utilisateurs(id_utilisateur) ON DELETE CASCADE,  
  INDEX idx_notification_utilisateur (id_utilisateur),  
  INDEX idx_notification_date (date_envoi)  
) ENGINE=InnoDB;
```

```
CREATE TABLE logs (  
  id_log BIGINT AUTO_INCREMENT PRIMARY KEY,  
  id_utilisateur BIGINT,  
  action VARCHAR(50) NOT NULL,  
  details TEXT,  
  date_action DATETIME DEFAULT CURRENT_TIMESTAMP,  
  ip_address VARCHAR(45),  
  FOREIGN KEY (id_utilisateur) REFERENCES utilisateurs(id_utilisateur) ON DELETE SET NULL,  
  INDEX idx_log_utilisateur (id_utilisateur),  
  INDEX idx_log_action (action),  
  INDEX idx_log_date (date_action)  
) ENGINE=InnoDB;
```

```
CREATE TABLE password_reset_tokens (  
  id BIGINT AUTO_INCREMENT PRIMARY KEY,  
  token VARCHAR(255) NOT NULL UNIQUE,  
  expiry_date DATETIME NOT NULL,  
  user_id BIGINT NOT NULL,  
  CONSTRAINT fk_token_user FOREIGN KEY (user_id) REFERENCES utilisateurs(id_utilisateur) ON DELETE  
CASCADE,  
  INDEX idx_token (token)  
) ENGINE=InnoDB;
```

7. Intégrité Référentielle et Stratégie de Suppression

Le choix des politiques ON DELETE pour chaque relation a fait l'objet d'une analyse métier spécifique afin d'équilibrer la simplicité de gestion et la préservation des données sensibles :

Relation	Politique	Justification métier
medecins / patients → utilisateurs	ON DELETE CASCADE	La suppression d'un compte utilisateur entraîne la suppression de son profil métier associé, les deux entités formant une unité logique indissociable.
disponibilites → medecins	ON DELETE CASCADE	Un créneau n'a pas de sens sans le médecin qui l'a déclaré.
rendez_vous → patients / disponibilites	ON DELETE CASCADE	Cohérence du planning : un rendez-vous ne peut exister sans son patient ni son créneau d'origine.
consultations → rendez_vous	ON DELETE RESTRICT	Empêche la suppression d'un rendez-vous ayant déjà donné lieu à une consultation, afin de préserver le dossier médical du patient de façon permanente.
notifications → utilisateurs	ON DELETE CASCADE	Les notifications n'ont de valeur que pour un utilisateur existant.
logs → utilisateurs	ON DELETE SET NULL	Les journaux d'audit sont conservés même après suppression d'un compte, pour des raisons de traçabilité légale et de sécurité.

7.1. Stratégie d'indexation

Des index secondaires ont été créés sur les colonnes fréquemment utilisées dans les clauses de filtrage (WHERE) et de jointure, en particulier : email (authentification), id_role, id_medecin / id_patient (jointures de planning), date_debut / date_heure (tri chronologique des créneaux et rendez-vous), ainsi que action et date_action sur la table d'audit pour accélérer les requêtes de supervision de l'administrateur.

7.2. Sécurité des données

- Aucun mot de passe n'est stocké en clair : hachage systématique via BCrypt (colonne mot_de_passe en VARCHAR(255)).
- Les jetons de réinitialisation de mot de passe sont à usage unique et possèdent une date d'expiration stricte (expiry_date).
- Les contraintes CHECK (chk_dates, chk_rdv_duree) empêchent l'insertion de données métier incohérentes directement au niveau du SGBD, en complément des validations applicatives.

- Le jeu de caractères utf8mb4 garantit la prise en charge complète des accents et symboles multilingues.

8. Conclusion

Le modèle de données de MedConnect ODC répond aux exigences de normalisation (3FN), d'intégrité référentielle et de traçabilité fixées par le cahier des charges. La séparation stricte entre l'entité d'authentification (utilisateurs) et les profils métier (patients, medecins) offre une base solide et évolutive, capable d'accueillir de futures fonctionnalités (messagerie, paiement en ligne, téléconsultation) sans remise en cause de l'architecture relationnelle existante.